



Despacho Laboral — Documentación Funcional del Sistema

Sistema: Aplicación web para la gestión de expedientes laborales de un despacho de abogados en Tijuana, México **Nombre del producto:** Conciliación Laboral Tijuana — Despacho Laboral **Tecnología:** Django 5.x · Tailwind CSS · HTMX · PostgreSQL/SQLite · Celery + Redis · Selenium **Última actualización:** Agosto de 2026

1. Qué hace el sistema

Despacho Laboral es una **plataforma de gestión de casos (CRM + flujo de trabajo legal)** para un despacho de abogados laboristas. Administra el ciclo de vida completo de una reclamación laboral — desde el primer contacto con el cliente, pasando por la etapa de conciliación prejudicial obligatoria ante el *Centro de Conciliación Laboral (CCL)* de Baja California, hasta el acuerdo (convenio) o la presentación de una **demanda laboral** formal.

Más allá del seguimiento de casos, la plataforma automatiza las partes más repetitivas y propensas a error de la práctica:

- **Cálculo de prestaciones laborales** (aguinaldo, prima vacacional, prima de antigüedad, indemnización constitucional, etc.) calculados automáticamente conforme a la Ley Federal del Trabajo (LFT).
- **Generación de documentos legales** a partir de plantillas HTML reutilizables (machotes) con inyección automática de datos.
- **Envío automatizado de solicitudes de conciliación** al portal oficial del gobierno de BC (`app.conciliacionbc.gob.mx`) — tanto del lado del servidor como mediante una extensión de Chrome complementaria.
- **Notificaciones por WhatsApp** a los clientes (manuales, automáticas por cambio de estado y recordatorios).
- **Un módulo financiero completo** (pagos, gastos, nómina, comisiones, caja diaria, préstamos entre socios, distribución de utilidades) organizado por oficina y por semana de trabajo.

2. Usuarios, roles y permisos

2.1 Roles (5)

Rol	Clave	Alcance
Superadmin	superadmin	Acceso total: admin de Django, todos los dashboards, matriz de permisos, gestión de usuarios
Administrativo	admin	Ve todos los casos, reportes de productividad, montos totales, exportaciones a Excel, acceso al admin de Django
Asesor	asesor	Solo ve y edita sus propios casos; dashboard personal con estadísticas
Abogada	abogada	Dashboard dedicado para la abogada del despacho (visión general de casos y documentos)
Finanzas	finanzas	Dashboard financiero y acceso al módulo de finanzas (cuenta como admin para permisos)

2.2 Capacidades del perfil (`UserProfile`)

Cada perfil de usuario cuenta además con:

- **`puede\_generar\_documentos`** — bandera que otorga acceso a los generadores de documentos legales (demanda, machotes, documentos legales). Desactivada por defecto.
- **`api\_token`** — token personal de API que usa la extensión de Chrome para autenticarse contra la API de la aplicación. Se genera automáticamente al crear el perfil y se puede regenerar (invalidando el anterior) desde la página **Extensión de Chrome**.

2.3 Auditoría de permisos (`PermisoAuditLog`)

Cada vez que un superadmin/admin modifica el rol de un usuario, sus permisos de documentos u otros privilegios, se registra una **entrada de auditoría**: a quién se modificó, quién lo modificó, la acción (`cambio_rol`, `cambio_docs`, `mixto`) y el detalle (ej: Rol: `asesor` → `admin`).

2.4 Módulo de superadmin

- **Dashboard de superadmin** — panel de administración de la plataforma.
- **Matriz de permisos** (`matriz_permisos`) — cuadrícula para gestionar el rol y el permiso de generación de documentos de cada usuario, con **exportación a Excel**.
- **Cargar datos demo** — siembra de datos de prueba con un clic (usuarios/casos).

2.5 Autenticación y seguridad

- Inicio/cierre de sesión y flujo completo de **restablecimiento de contraseña** por correo (SMTP configurable; backend de consola por defecto en desarrollo).
 - Redirección automática al dashboard correspondiente según el rol tras iniciar sesión.
 - Endurecimiento en producción: redirección HTTPS, cookies seguras, detección automática del dominio de Railway, `DEBUG` / `SECRET_KEY` / `ALLOWED_HOSTS` controlados por entorno.
-

3. Gestión de casos (Módulo Expedientes)

3.1 Clientes (`Cliente`)

Ficha de cliente enriquecida diseñada específicamente para la conciliación laboral:

- **Identidad:** nombre, **CURP** (único), RFC, teléfono, WhatsApp, correo, fecha de nacimiento, género.
- **Domicilio particular:** calle, número, código postal, colonia — se ensambla en `direccion_completa`.
- **Datos del patrón:** empresa, actividad económica, teléfono, razón social, domicilio completo y referencias.
- **Tipo de persona del patrón** (física / moral) — se usa en el formulario del portal de conciliación.
- **Datos laborales:** puesto, **salario mensual**, periodo de pago (diario/semanal/ quincenal/mensual), horas semanales, jornada (diurna/nocturna/mixta), fecha de ingreso, fecha de salida.
- **Fuente de captación:** cómo se enteró el cliente del despacho (Facebook, Google, recomendación, WhatsApp, TikTok, TV/radio, volante...).
- **Asignación de oficina** (Plaza Patria, Otay, CLT).
- **Seguimiento de asesoría gratuita:** si se ofreció la asesoría gratuita semanal, si se agendó y en qué fecha.
- Indexado para búsquedas rápidas por CURP, nombre y fecha de asesoría.

3.2 Expedientes (`Expediente`)

- **Numeración automática** `AAAA-####` (año + secuencial), única y no editable.
- Asignado a un **asesor** (asesor jurídico).
- **Montos:** monto reclamado (`monto_reclamado`) y monto de convenio (`monto_convenio`).
- **Programación de audiencias** con fecha/hora y resultado registrado.
- **Datos de conciliación:** tipo de despido (justificado/injustificado/voluntario/ rescisión/otro), prestaciones reclamadas, **folio** y fecha de trámite.
- **Seguimiento:** fecha de próxima acción, notas internas, prioridad (baja/media/alta).
- **Interruptor de automatización WhatsApp por caso** (`notificar_whatsapp_auto`).

Máquina de estados con transiciones validadas

```
nuevo → solicitud → citatorio → audiencia → no_notificado ┐
                                                    ↳ reprogramacion ┘
                                                    ↳ convenio ───┐
                                                    ↳ sin_conciliacion ┘ → demanda → audiencia/convenio/cerrado
                                                    ↳ cerrado (desde cualquier estado)
```

El modelo **rechaza transiciones ilegales** (ej. saltar de `nuevo` a `audiencia`) con un mensaje claro que enumera las transiciones permitidas. Cada estado tiene un color asociado para las insignias de la interfaz.

3.3 Documentos (`Documento`)

- Subida de PDFs/imágenes por caso con clasificación por tipo (INE, contrato, evidencia, screenshot/captura, citatorio, PDF, otro) y descripción; se registra quién subió y cuándo.
- Organizados en disco por año/mes.

3.4 Notas e historial de actividad

- **Notas** — notas libres por caso con autor y fecha.

- **Movimientos** — rastro de auditoría automático de cada acción: creación, cambio de estado, actualización, subida de documento, nota agregada, resultado de audiencia (quién, cuándo, qué cambió).

3.5 Notificaciones (campana)

Notificaciones internas para el usuario: transferencias, avisos, mensajes del sistema, recordatorios. Cada notificación puede llevar un enlace; se pueden marcar como leídas individualmente o **todas de una vez**.

3.6 Solicitudes de transferencia entre asesores

Cuando un asesor no puede asistir a una audiencia (conflicto de horario, etc.):

- Solicita una **transferencia** del caso, opcionalmente sugiriendo un asesor destino, con un motivo obligatorio.
- El área administrativa revisa la solicitud, opcionalmente **reasigna** a otro asesor y aprueba o rechaza con un comentario.
- Estados: pendiente → aprobada / rechazada / cancelada. Las notificaciones mantienen informados a todos.

3.7 Avisos obligatorios del administrador

- El administrador crea avisos/pendientes con título, contenido, prioridad y **fecha de vencimiento opcional** (se ocultan automáticamente al vencer).
- Se muestran en el dashboard de todos los asesores y administradores; los usuarios pueden marcarlos como **leídos/entendidos** (se registra por usuario mediante M2M).

3.8 Búsqueda global y filtros

- **Búsqueda global** en expedientes, clientes, folios y empresas.
- Filtros avanzados en la lista de expedientes; los asesores solo ven sus propios casos.

3.9 Calendario

- **Calendario de audiencias** (`calendario`) que agrega todas las audiencias programadas.

3.10 Reportes y exportaciones

- **Exportación a Excel** de expedientes.
- **Reportes administrativos** (`reportes_admin`) — productividad y totales por asesor.
- **PDF por expediente** (`pdf_expediente`) generado con WeasyPrint.

4. Dashboards (por rol)

Dashboard	Qué muestra
Asesor	KPIs personales, sus casos por estado, próximas acciones/audiencias, tareas pendientes, avisos
Abogada	Vista orientada a la abogada del despacho (casos y documentos)
Admin	Todos los casos, reportes de productividad, montos totales, avisos, acciones de exportación
Finanzas	KPIs financieros (ver módulo de Finanzas)
Superadmin	Panel de administración de la plataforma + matriz de permisos

5. Motor de Cálculo Laboral (Cálculo Laboral)

5.1 Reglas legales configurables (LegalConfig)

Todos los parámetros legales se almacenan en la base de datos y son editables desde el admin de Django — **sin necesidad de tocar código cuando cambian las leyes**:

- **UMA** diaria, salario mínimo general y **salario mínimo de la zona libre de la frontera norte (ZLF)**.
- **Días de aguinaldo** (mínimo legal: 15).
- **% de prima vacacional** (mínimo 25%).
- **Prima de antigüedad**: días por año (12) + tipo de tope salarial (2× UMA / 2× SM / 2× SM frontera) y múltiplo.
- **Días de indemnización** (3 meses = 90).
- Solo una configuración puede estar **activa** a la vez; al guardar una se desactivan las demás.

5.2 Tabla de vacaciones (Reforma LFT 2023)

El sistema implementa la tabla de vacaciones vigente en México (año 1 → 12 días, +2 por año hasta 20, después +2 cada 5 años), extrapolando más allá de la tabla cuando es necesario.

5.3 Conceptos calculados

Concepto	Artículo	Tipo
Aguinaldo proporcional	Art. 87 LFT	automático
Vacaciones proporcionales	Art. 76 LFT	automático
Prima vacacional	Art. 80 LFT	automático
Prima de antigüedad (con indicador de tope)	Art. 162 LFT	automático
Indemnización constitucional (90 días)	Art. 50 LFT	automático
Indemnización 20 días por año	Art. 50-II LFT	automático
Vacaciones vencidas (días manuales)	Art. 76 LFT	semiautomático
Horas extras (horas manuales)	Art. 66-68 LFT	semiautomático
Salarios devengados (monto manual)	Art. 48 LFT	manual
Días festivos (días manuales)	Art. 75 LFT	semiautomático

5.4 Cálculo por caso (CalculoLaboral)

- Un cálculo por expediente que guarda una **instantánea de los datos de entrada** para conservar el histórico.
- **Casillas de verificación** que permiten al asesor elegir qué conceptos incluir (con valores por defecto sensatos).
- **Sustitución manual** de los días de vacaciones realmente adeudados (cuando algunos años ya se pagaron o disfrutaron) — se muestra con el indicador **override aplicado**.
- Muestra salario diario, días trabajados, años de servicio, desglose por concepto con referencia legal, advertencias de tope aplicado y **total**.
- **Recálculo automático** cuando cambian los datos del cliente/expediente o las reglas legales (`recalcular`).
- **Simulación rápida** (`simulacion-rapida`) — estimación instantánea para un prospecto sin crear un caso: captura salario + fechas y obtén el desglose completo.

6. Generación de Documentos Legales

6.1 Machotes (plantillas HTML reutilizables)

- Plantillas categorizadas como: **Demanda Laboral, Carta Finiquito, Convenio, Solicitud, Citatorio, Otro**; con jurisdicción (Federal / Estatal BC / Ambas), tipo de despido opcional, ícono, bandera de activo y orden.
- Los machotes **favoritos** aparecen primero en el editor.
- El cuerpo de la plantilla es HTML con **marcadores** como `{{ nombre }}`, `{{ empresa }}`, etc.

6.2 Motor de inyección de marcadores (`marcadores.py`)

- ~40 marcadores con **datos reales del caso**: cliente, empresa, expediente, fechas (en español, ej. "1 de enero de 2024"), formato de salarios **y prestaciones calculadas** (aguinaldo, vacaciones, indemnización...) obtenidas automáticamente del motor de cálculo.
- Los datos faltantes se muestran como marcadores visibles (`[CURP]` , `[MONTO]` , ...) en lugar de espacios en blanco o errores — el usuario sabe exactamente qué completar.
- Verificador de completitud**: lista qué campos están completos/incompletos por sección (Cliente, Empleo, Empresa, Expediente) con **enlaces directos de edición** y un porcentaje de completitud.

6.3 Flujo de documentos

- Elegir un machote del catálogo (o desde el expediente).
- Preparar** — ver qué datos faltan y corregirlos con un clic.
- Editar/vista previa** del documento renderizado en el navegador.
- Generar/descargar** como PDF.
- Opcionalmente **guardar el documento editado como nuevo machote** (la biblioteca de plantillas del equipo crece de forma orgánica).

6.4 Importar machotes desde Word

- Los archivos `.docx` (ej. una carpeta con demandas existentes) se convierten en plantillas HTML mediante el comando `importar_machotes` o la importación web, conservando los marcadores cuando es posible.
- La biblioteca de plantillas se administra desde el admin de Django o desde la interfaz de machotes.

6.5 Generador de Demanda Laboral (`demanda_generator.py`)

- Genera una **demanda laboral mexicana profesional** (Word `.docx`) con todos los datos del caso, cálculos integrados y formato listo para imprimir y firmar.
- Un **asistente paso a paso** (`demanda_asistente`) guía por las secciones de la demanda; un **generador directo** produce el documento en un solo clic.
- Se descarga como `.docx` para edición/firma.

7. Automatización de Conciliación (Portal de Baja California)

El sistema se integra con el portal oficial `app.conciliacionbc.gob.mx` en **dos modos**.

7.1 Automatización del lado del servidor (`conciliacion\_automation.py` + Celery)

- Llena el flujo real de varias fases del portal: aviso de privacidad → industria → fecha del conflicto y objeto → trabajador (solicitante) → empresa (citado) → descripción → envío.
- Se ejecuta **headless** (servidor) o en **modo visible/debug**.
- Se ejecuta de forma asíncrona mediante un **worker de Celery** (o fallback con threads si no hay Redis) para que las peticiones HTTP nunca expiren.
- Página de progreso en vivo (`conciliacion_procesando`) que muestra **capturas de pantalla del navegador** y el tiempo transcurrido; soporta reintentos.
- Al completarse: captura el **folio**, guarda el **PDF del acuse** en el expediente y marca la TareaConciliacion como completada con bitácoras completas.
- Comando de gestión `enviar_solicitud_conciliacion` para envíos por lotes/CLI.

7.2 Parser de acuse (`acuse\_parser.py`)

Cuando hay un PDF de acuse oficial disponible (descargado automáticamente o subido manualmente), el sistema **extrae su texto con PyMuPDF** y autocompleta el expediente:

- Folio (ej. `TIJ/26427/2026`), fecha de solicitud, solicitante, empresa citada, fecha del conflicto, objeto, unidad de conciliación.
- **Mapa el objeto de la conciliación → tipo de despido** (ej. "Despido" → injustificado).
- **Pantalla de vista previa** que compara los valores detectados con los actuales (marcando los nuevos o los que difieren) y permite al asesor **confirmar** antes de guardar.

7.3 Extensión de Chrome — "Conciliación BC Asistente"

Para cuando se detecta navegación automatizada o hay que resolver un CAPTCHA a mano, la **extensión de Chrome complementaria (Manifest V3)** llena el formulario del portal **en el navegador del propio asesor**:

- El asesor hace clic en *Enviar a Conciliación → Desde mi navegador (Extensión de Chrome)*.
- La extensión muestra las tareas pendientes (número de expediente, cliente, empresa, CURP) en su popup.
- 🚀 **Llenar en el portal** abre el portal con el formulario **ya llenado** (aviso de privacidad, industria, fechas, trabajador, empresa, descripción) mediante un content script — el portal ve una sesión dirigida por una persona.
- El asesor revisa, resuelve CAPTCHAs en vivo y hace clic en *Enviar solicitud* él mismo.
- 🔍 **Ya envié, detectar acuse** — la extensión detecta el folio, descarga el PDF del acuse, toma una captura de pantalla y **reporta todo a la API de la aplicación** (autenticada con el token personal del usuario), adjuntando el PDF al expediente.
- Pantalla de opciones que guarda la URL de la app + token y verifica la conexión.
- El paquete de la extensión se puede **descargar como .zip** desde la app (`/extension/descargar/`), listo para cargarse descomprimido en Chrome.

7.4 Fallbacks manuales

- Subir el PDF del acuse manualmente (`subir-conciliacion`) y luego previsualizar/ confirmar los datos parseados.
 - Descargar un **formulario de conciliación prellenado (PDF)** para presentarlo manualmente en el centro.
 - Subir el acuse mediante el flujo normal de documentos.
-

8. Integración con WhatsApp

Dos métodos de envío:

1. **Deep links (wa.me)** — gratis; abre WhatsApp con un mensaje pre-llenado.
2. **API de WhatsApp de Twilio** — mensajes enviados desde el servidor (requiere credenciales de Twilio); cae automáticamente al deep link cuando no está configurado.

Funcionalidades

- **Plantillas** para: recordatorio de audiencia, citatorio, seguimiento de convenio, seguimiento general, solicitud de documentos y mensajes personalizados — con variables (`{cliente}`, `{fecha}`, `{asesor}` ...).
 - **Mensajes automáticos por cambio de estado**: cada estado del expediente tiene un mensaje configurado (caso nuevo, solicitud creada, citatorio, audiencia, no notificado, reprogramación, convenio, sin conciliación, demanda, cerrado). Interruptor por caso para activar/desactivar.
 - **Historial de mensajes** por caso con estado (pendiente/enviado/fallido), canal y bitácoras de error.
 - Normalización de números mexicanos (agrega el código de país 52 y limpia formatos comunes).
 - Comando de gestión `enviar_whatsapp_automatico` para vaciar mensajes pendientes (con opciones `--dry-run` y `--send-twilio`).
-

9. Importaciones y utilidades

9.1 Importación CLT (citas desde Excel)

El archivo `CLT.xlsx` (citas de conciliación pre-programadas del CCL) se importa como **expedientes con audiencias programadas**:

- Importación por hoja (ej. pestañas mensuales), **creación automática de usuarios asesores** encontrados en el archivo si es necesario, y opciones de limpieza para datos de prueba.
- La CURP es opcional al importar y se captura después (para que los números fluyan rápido).

9.2 Catálogo de empresas (‘Empresa’)

- Importación desde `Empresas y Domicilios.xlsx` — **idempotente**, nombres normalizados (mayúsculas, sin acentos, espacios compactados) para que reimportar un archivo en crecimiento nunca duplique registros.
- Tipo de persona detectado automáticamente; el domicilio desglosado se usa para el **autocompletado en el formulario de cliente** (endpoint AJAX `empresas/buscar/`).

9.3 Generador de CURP

Herramienta independiente para **generar una CURP válida** a partir de los datos personales (nombre, fecha de nacimiento, género, estado) — ahorra búsquedas manuales al dar de alta clientes.

9.4 Ajustes del usuario

Página de ajustes por usuario: información del perfil, teléfono, preferencia de automatización de WhatsApp, gestión del token de API y configuración de la extensión.

10. Módulo de Finanzas

Organizado por **oficina** (sucursal) y por **semana de trabajo** para reportes operativos.

10.1 Oficinas

Catálogo de oficinas del despacho con dirección, teléfono, responsable y bandera de activa. Cada oficina tiene su propia operación de ingresos/gastos/caja diaria.

10.2 Semanas de trabajo

El sistema financiero gira alrededor de **semanas** (lunes–domingo): la semana actual se crea automáticamente; cada movimiento se puede atribuir a una semana. Las semanas se pueden abrir/cerrar. Totales por semana: **ingresos** (caja + pagos de convenios), **gastos** (caja + gastos + nómina) y **balance**, calculados dinámicamente.

10.3 Pagos de convenios (`SettlementPayment`)

Pagos recibidos por acuerdos (convenios): fecha, cliente, expediente, monto, forma de pago (efectivo/transferencia/cheque/tarjeta crédito/tarjeta débito/depósito/otro), oficina, notas y responsable del registro.

10.4 Gastos (`Expense`)

Gastos operativos por oficina con categoría (renta, luz, agua, internet, teléfono, papelería, publicidad, sueldos, gasolina, mantenimiento, impuestos, equipo, muebles, seguros, honorarios, otro), proveedor, folio fiscal/factura y responsable del registro.

10.5 Convenios y honorarios (`Agreement` / `Honorario`)

- **Convenios** con el cliente: monto, estado (pendiente de firma → firmado → pagado / pagado parcialmente / cancelado), responsable, oficina y **total de honorarios automático**.
- **Honorarios**: una o varias partidas por convenio con un **% de tarifa** (25/30/35/40/50%); el monto se calcula automáticamente ($\text{convenio} \times \% \div 100$), con fechas estimada/pagada y sumas de pagado/pendiente agregadas en el convenio.

10.6 Comisiones (`Commission`)

Comisiones de asesores sobre convenios cerrados: % configurable, **monto calculado automáticamente**, estado (pendiente/pagada/cancelada), fecha de pago, oficina y responsable del registro.

10.7 Empleados y nómina (`Employee` / `Payroll`)

- Catálogo de empleados con puesto (administrativo, asesor jurídico, supervisor, contador, recepcionista, auxiliar, director...), periodo de pago, salario, oficina y bandera de activo.
- **Pagos de nómina** por período: tipo de período (semanal/quincenal/mensual/extraordinario/aguinaldo/prima vacacional/bono), rango del período, salario pagado, descuentos (ISR/IMSS) y **total calculado automáticamente** ($\text{salario} - \text{descuentos}$), oficina y responsable del registro.

10.8 Caja diaria (`CashMovement`)

Movimientos de caja por oficina y fecha: **ingresos** (pago de convenio, pago de cliente, anticipo, devolución, otro) o **egresos** (papelería, gasolina, renta, luz, agua, internet, teléfono, viáticos, comisiones, honorarios, otro). La coherencia entre categoría y tipo se **valida** (ej. un ingreso no puede tener categoría de egreso). El campo de referencia vincula el movimiento al expediente/factura.

10.9 Socios, préstamos y distribución de utilidades

- **Socios (`Partner`)**: % de participación por socio para el reparto de utilidades.

- **Préstamos entre socios (`PartnerLoan`)**: registro formal de préstamos entre socios (origen → destino, monto, concepto, estado, saldo pendiente), que reemplaza las notas informales tipo "le debo a..."
- **Distribución de utilidades (`ProfitDistribution` + `PartnerProfit`)**:
- Calcula automáticamente la **utilidad neta** = monto del convenio – honorarios – comisiones – retenciones – gastos relacionados.
- La distribuye entre los socios según su % de participación (**participación individual calculada automáticamente**), con estados (borrador → distribuida → confirmada/cancelada).
- **Resúmenes acumulados por socio** (generada / pagada / pendiente) que se actualizan automáticamente al confirmar distribuciones.

10.10 Dashboard financiero

- KPIs por oficina y globales; **flujo de caja mensual** con gráfica alimentada por el endpoint JSON `api_flujo_mensual`; **exportación a Excel** de los totales del dashboard.
- **Reporte de convenios** (`reporte_convenios`) con exportación a Excel.

11. Comandos de gestión (CLI)

Comando	Propósito
<code>crear_usuarios_prueba</code>	Crear usuarios de prueba (1 superadmin, 4 admins, 15 asesores)
<code>seed_datos / seed_clientes_prueba</code>	Sembrar casos demo (10 escenarios laborales / clientes completos con cálculos)
<code>enviar_recordatorios</code>	Enviar recordatorios de próximas acciones/audiencias (<code>--days</code> , <code>--dry-run</code>)
<code>enviar_solicitud_conciliacion</code>	Enviar solicitudes de conciliación al portal de BC desde CLI (<code>--headless</code>)
<code>enviar_whatsapp_automatiko</code>	Vaciar mensajes automáticos de WhatsApp pendientes (<code>--dry-run</code> , <code>--send-twilio</code>)
<code>importar_clt</code>	Importar citas de CLT.xlsx como expedientes con audiencias
<code>importar_empresas</code>	Importar el catálogo de empresas y domicilios (idempotente)
<code>importar_machotes</code>	Convertir plantillas <code>.docx</code> en machotes (<code>--reload</code> , <code>--file</code>)
<code>migrate_sqlite_to_pg</code>	Migración única de datos de SQLite a PostgreSQL

12. Resumen de integraciones

Servicio	Uso	Configuración
Selenium	Automatización del portal (headless/debug)	incluido
Celery + Redis	Tareas en segundo plano (automatización de conciliación); fallback con threads	REDIS_URL
Twilio	Envío por API de WhatsApp (opcional)	variables de entorno TWILIO_*
SMTP	Correos de restablecimiento de contraseña (backend de consola en desarrollo)	variables de entorno EMAIL_*
WeasyPrint	Generación de PDFs (expediente, documentos)	incluido
python-docx	Generación de demandas en Word (.docx)	incluido
PyMuPDF (fitz)	Extracción de texto de los PDFs de acuse	incluido
openpyxl	Importaciones (CLT, empresas) y exportaciones a Excel	incluido
API de la extensión de Chrome	Endpoints REST autenticados con token para la extensión	api_token personal
Railway	Despliegue; detecta automáticamente dominio, PostgreSQL y Redis	variables de entorno

13. Arquitectura y notas técnicas

- **Backend:** Django 5.x, locale `es-mx`, zona horaria `America/Mexico_City`.
- **Base de datos:** SQLite (desarrollo) / PostgreSQL (producción) vía `DATABASE_URL`; detección automática del entorno de Railway (`PGHOST` ...).
- **Frontend:** Tailwind CSS (CDN) + **HTMX** para interacciones dinámicas (sin framework JS pesado); diseño responsive.
- **Archivos estáticos:** Whitenoise (`CompressedManifestStaticFilesStorage`) para servir en producción sin Nginx.
- **Trabajo en segundo plano:** Celery cuando hay Redis, **fallback con threads** en caso contrario (degradación elegante para desarrollo local).
- **Auditabilidad:** cada entidad clave (pagos, gastos, nómina, caja, préstamos, distribuciones, comisiones) registra `registrado_por` / `creado_por`.
- **Despliegue:** Dockerfile, configuración de Railway, entrypoint (levanta el worker de Celery automáticamente cuando existe Redis), receta de Nginx + Gunicorn para VPS (droplet) y guía de despliegue para clientes.

14. Pruebas

La suite incluye **48 pruebas** (expedientes, finanzas, automatización) que cubren la validación de transiciones de estado, los cálculos financieros y el script de verificación `verify_demandas_calculos.py` (usado por CI).

Ejecución:

```
uv run python manage.py test
```

15. Inicio rápido (desarrollo)

```
uv sync                                # instalar dependencias
uv run python manage.py migrate        # migraciones de BD
uv run python manage.py crear_usuarios_prueba
uv run python manage.py runserver
```

Accesos de prueba: `superadmin/Admin123!` · `admin1/Admin1!` · `asesor1/Asesor1!`